CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with this repository.

Project

Little Secret is a browser port of Exploding Kittens' social-deduction game. The Go backend is authoritative and communicates with the React client over a single WebSocket endpoint (`/ws`). There is no REST API for gameplay.

Current implementation status:

- Full main loop is implemented: `WaitingForPlayers -> Info -> SubmitClue -> Vote -> RevealRole -> ... -> GameOver`
- Round progression, tie handling (random pick with tied list metadata), clue validation, spy guess outcomes, and winner reasons are wired
- Crash recovery and rejoin are implemented via event log + snapshots

Official rules asset:

- PDF lives at `public/Little_Secret_US_FLYER.pdf` (app URL: `/Little_Secret_US_FLYER.pdf`)

Commands

Dev loops

Run DB + app stack in separate shells:

- `yarn supabase:start` (or `make db-run`) to start local Supabase/Postgres
- `docker compose watch` for containerized backend + frontend hot reload
- Alternative host-native loops:
  - `make go-srv` for backend
  - `yarn dev` for frontend

Quality checks

- Go: `make go-fmt` , `make go-lint` , `make go-test`
- Web: `yarn lint` , `yarn fmt` , `yarn build`
- Combined: `make fmt` , `make lint` , `make test`

Test commands

- Unit: `yarn test:unit`
- Storybook interaction: `yarn test:storybook`
- E2E all: `yarn test` or `make yarn-test`
- E2E journey only: `yarn test:e2e:journey`
- E2E with external dev server: `PLAYWRIGHT_SKIP_WEB_SERVER=1 yarn test:e2e:external`

Architecture

Backend

Entrypoints:

- `cmd/srv/main.go` (production)
- `cmd/cli/main.go` (`srv local` for local dev)

Server wiring:

- `internal/server.ServeWeb` creates the router
- gameplay route is `GET /ws` handled by `GameService.HandleWebSocket`

Game service:

- `internal/game/service/` owns room connections and message routing
- `events.go` handles game event mutations
- `sockets.go` handles per-connection read loop
- outbound payloads should include `gameState` when public state changes

Persistence and recovery:

- Event store interface: `internal/game/service/eventstore.go`
- Postgres event store: `internal/game/service/eventstore_pg.go`
- Snapshot recovery on boot: `internal/game/service/recover.go`
- Rejoin events supported: `rejoin` , `rejoined` , `rejoinFailed`

Domain:

- `internal/game/` models game state and rules
- roles/word data from embedded JSON in `internal/game/data/`
- winner reasons include:
  - `allImpostersOut`
  - `impostersSurvived`
  - `spyGuessedWord`
  - `twoPlayersLeft`

Frontend

- `src/components/GameStateProvider.tsx` owns socket lifecycle and merged state
- `src/components/Game.tsx` renders phase components by `phase`
- Phase components include Lobby, Info, Clue, Vote, RevealRole, GameOver
- GameOver includes leave-room behavior
- Rules UI is in `src/App.tsx` with written rules + links to PDF/video

Session/rejoin behavior:

- Client stores `{roomCode, playerID}` in localStorage
- On WS open: attempts `rejoin` if session exists, otherwise `playerConnected`
- On `rejoinFailed` : clears session and restarts as fresh connection

E2E contract: preserve these selectors

Playwright relies on stable selectors and data attributes. Do not rename/remove without updating tests:

- Lobby/setup:
  - `#host-btn` , `#player-btn`
  - `#room-code` , `#room-code-input` , `#room-code-submit`
  - `#start-game-btn`

- Info:
  - `#player-word`, `#confirm-word-btn`
- Clue/Vote/Reveal/GameOver:
  - `#clue-input`, `#player-clue-submit`
  - `#vote-header`, `[data-testid^="vote-for-"]`
  - `#reveal-continue-btn`
  - `#game-over-summary`, `#game-over-leave-btn`
- Hidden E2E state surface:
  - `[data-testid="e2e-root"]` with `data-phase` and `data-player-id`
  - `#e2e-players-json`

Cross-cutting rules

- Client/server event parity is required:
  1. `internal/game/events.go` event constant
  2. `internal/game/service/events.go` handler switch
  3. `src/game/events.ts` enum
  4. `src/components/GameStateProvider.tsx` inbound handling
- Keep phase enum parity between Go and TS
- If changing room/session semantics, validate rejoin and GameOver leave flow

Known remaining items

From current TODO scope:

- Apply Supabase migration before relying on persisted event logs in live DB
- Re-run journey tests after backend restarts in environments where rejoin handler is stale
- Deferred/stretch: Secret Weapons and full tie-defense + revote flow